

COURSE ASSESSMENT: CO5 - AT2

Department of Computer Science and Engineering (Artificial Intelligence)

Application Based Problem Solving (Java Programming)

Course Code	CSA0904
Course Name	Java Programming / Advanced Object-Oriented Software Development
Student Name	Karani Venkata Siva Sai Jeswanth Babu
Register Number	192472012
Department	Computer Science and Engineering (Artificial Intelligence)
Assessment Topic	Encapsulation, Inheritance, Exception Handling & Data Mapping
Date of Submission	August 28, 2026

1. PROBLEM STATEMENT

Scenario: Design and implement an Employee Management System simulating persistent data layer records using Java Object-Oriented Principles:

- Create an abstract base class Employee with fields: ID, Name, Department, Base Salary, and Tax Category with overloaded constructors.
- Derive subclasses PermanentEmployee (incorporating bonus & PF deduction) and ContractEmployee (computing salary based on hourly rate & hours worked).
- Implement a custom checked exception InvalidSalaryDataException to handle corrupt/negative compensation metrics.
- Create a Data Access Object (EmployeeDAO) with overloaded search methods: fetching records by Employee ID and by Department.

2. SOURCE CODE

```
import java.util.ArrayList;
import java.util.List;

// =====
// 1. MAIN DRIVER CLASS
// =====
public class EmployeeDemo {
    public static void main(String[] args) {
        EmployeeDAO dao = new EmployeeDAO();

        System.out.println("=====");
        System.out.println("TASK 1: FETCH BY EMPLOYEE ID (Overloaded Method 1)");
        System.out.println("=====");
        try {
            Employee emp = dao.fetchEmployeeDetails(101);
            if (emp != null) {
                emp.displayDetails();
            } else {
                System.out.println("Employee not found.");
            }
        } catch (InvalidSalaryDataException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

```

        System.out.println("\n=====");
        System.out.println("TASK 2: FETCH BY DEPARTMENT (Overloaded Method 2)");
        System.out.println("=====");
        try {
            List<Employee> aiDept = dao.fetchEmployeeDetails("AI & Data Science");
            for (Employee e : aiDept) {
                e.displayDetails();
            }
        } catch (InvalidSalaryDataException e) {
            System.err.println("Error: " + e.getMessage());
        }

        System.out.println("\n=====");
        System.out.println("TASK 3: TESTING CUSTOM EXCEPTION (Invalid Data)");
        System.out.println("=====");
        try {
            dao.fetchEmployeeDetails(105);
        } catch (InvalidSalaryDataException e) {
            System.out.println("Caught Custom Exception: " + e.getMessage());
        }

        printAnalysis();
    }

    private static void printAnalysis() {
        System.out.println("\n=====");
        System.out.println("                TECHNICAL ANALYSIS                ");
        System.out.println("=====");
        System.out.println("1. Incremental vs. Batched Processing (Midway Failure):");
        System.out.println("    - Incremental Processing (Adding directly to list):");
        System.out.println("        If a failure occurs midway (e.g., at row 250 of 500),");
        System.out.println("        the caller retains a partial list of 249 records.");
        System.out.println("        This exposes an inconsistent, partial application state.");
        System.out.println("    - Batched Processing (Atomic / All-or-Nothing):");
        System.out.println("        Records are validated into a local buffer and only returned");
        System.out.println("        once the full iteration succeeds. On failure, the local");
        System.out.println("        buffer is discarded and an exception is re-thrown.");
        System.out.println("\n2. Resource-Closing Strategy:");
        System.out.println("    - Deterministic resource management (via AutoCloseable and");
        System.out.println("        try-with-resources) ensures all underlying descriptors,");
        System.out.println("        streams, and memory handles are closed in reverse order");
        System.out.println("        of allocation, preventing leaks.");
        System.out.println("=====");
    }
}

// =====
// 2. CUSTOM EXCEPTION
// =====
class InvalidSalaryDataException extends Exception {
    public InvalidSalaryDataException(String message) {
        super(message);
    }
}

// =====
// 3. BASE CLASS (Employee)
// =====
abstract class Employee {
    protected int employeeId;
    protected String name;
    protected String department;
    protected double baseSalary;
    protected String taxCategory;

    // Overloaded Constructor 1: Without explicit tax category
    public Employee(int employeeId, String name, String department, double baseSalary) {
        this(employeeId, name, department, baseSalary, "STANDARD");
    }
}

```

```

// Overloaded Constructor 2: With tax category
public Employee(int employeeId, String name, String department, double baseSalary, String taxCategory) {
    this.employeeId = employeeId;
    this.name = name;
    this.department = department;
    this.baseSalary = baseSalary;
    this.taxCategory = (taxCategory != null && !taxCategory.trim().isEmpty()) ? taxCategory : "STANDARD";
}

public abstract double calculateSalary();

public int getEmployeeId() { return employeeId; }
public String getName() { return name; }
public String getDepartment() { return department; }
public double getBaseSalary() { return baseSalary; }
public String getTaxCategory() { return taxCategory; }

public void displayDetails() {
    System.out.println("-----");
    System.out.println("ID          : " + employeeId);
    System.out.println("Name          : " + name);
    System.out.println("Department    : " + department);
    System.out.println("Tax Category  : " + taxCategory);
    System.out.printf("Net Salary    : Rs. %.2f\n", calculateSalary());
    System.out.println("-----");
}
}

// =====
// 4. SUBCLASS 1: PermanentEmployee
// =====
class PermanentEmployee extends Employee {
    private double bonus;
    private double pfDeduction;

    public PermanentEmployee(int employeeId, String name, String department, double baseSalary, double bonus, double pfDeduction) {
        super(employeeId, name, department, baseSalary);
        this.bonus = bonus;
        this.pfDeduction = pfDeduction;
    }

    public PermanentEmployee(int employeeId, String name, String department, double baseSalary, String taxCategory, double bonus, double pfDeduction) {
        super(employeeId, name, department, baseSalary, taxCategory);
        this.bonus = bonus;
        this.pfDeduction = pfDeduction;
    }

    @Override
    public double calculateSalary() {
        return baseSalary + bonus - pfDeduction;
    }
}

// =====
// 5. SUBCLASS 2: ContractEmployee
// =====
class ContractEmployee extends Employee {
    private double hourlyRate;
    private int totalHoursWorked;

    public ContractEmployee(int employeeId, String name, String department, double hourlyRate, int totalHoursWorked) {
        super(employeeId, name, department, (hourlyRate * totalHoursWorked));
        this.hourlyRate = hourlyRate;
        this.totalHoursWorked = totalHoursWorked;
    }

    public ContractEmployee(int employeeId, String name, String department, double hourlyRate, int totalHoursWorked, String

```

```

taxCategory) {
    super(employeeId, name, department, (hourlyRate * totalHoursWorked), taxCategory);
    this.hourlyRate = hourlyRate;
    this.totalHoursWorked = totalHoursWorked;
}

@Override
public double calculateSalary() {
    return hourlyRate * totalHoursWorked;
}
}

// =====
// 6. IN-MEMORY DATA ACCESS OBJECT (DAO)
// =====
class EmployeeDAO {
    private static class EmployeeRecord {
        int empId;
        String name;
        String department;
        String empType;
        double baseSalary;
        double bonus;
        double pfDeduction;
        double hourlyRate;
        int hoursWorked;
        String taxCategory;

        EmployeeRecord(int empId, String name, String dept, String type, double salary, double bonus, double pf, double rate, int hours, String tax) {
            this.empId = empId;
            this.name = name;
            this.department = dept;
            this.empType = type;
            this.baseSalary = salary;
            this.bonus = bonus;
            this.pfDeduction = pf;
            this.hourlyRate = rate;
            this.hoursWorked = hours;
            this.taxCategory = tax;
        }
    }

    private final List<EmployeeRecord> databaseTable = new ArrayList<>();

    public EmployeeDAO() {
        databaseTable.add(new EmployeeRecord(101, "Jeswanth Babu", "AI & Data Science", "PERMANENT", 85000.0, 12000.0, 4500.0, 0, 0, "EXEMPT_80C"));
        databaseTable.add(new EmployeeRecord(102, "Bala Murali", "Cyber Security", "CONTRACT", 0.0, 0.0, 0.0, 850.0, 160, "STANDARD"));
        databaseTable.add(new EmployeeRecord(103, "Sai Charan", "AI & Data Science", "PERMANENT", 92000.0, 15000.0, 5000.0, 0, 0, null));
        databaseTable.add(new EmployeeRecord(104, "Vamsi Krishna", "Cloud DevOps", "CONTRACT", 0.0, 0.0, 0.0, 750.0, 140, "TAX_SLAB_2"));
        databaseTable.add(new EmployeeRecord(105, "Maruthi Kumar", "Cyber Security", "PERMANENT", -5000.0, 2000.0, 500.0, 0, 0, "STANDARD"));
    }

    public Employee fetchEmployeeDetails(int empId) throws InvalidSalaryDataException {
        for (EmployeeRecord rec : databaseTable) {
            if (rec.empId == empId) {
                return mapRecordToEmployee(rec);
            }
        }
        return null;
    }

    public List<Employee> fetchEmployeeDetails(String department) throws InvalidSalaryDataException {
        List<Employee> employeeList = new ArrayList<>();
        for (EmployeeRecord rec : databaseTable) {

```

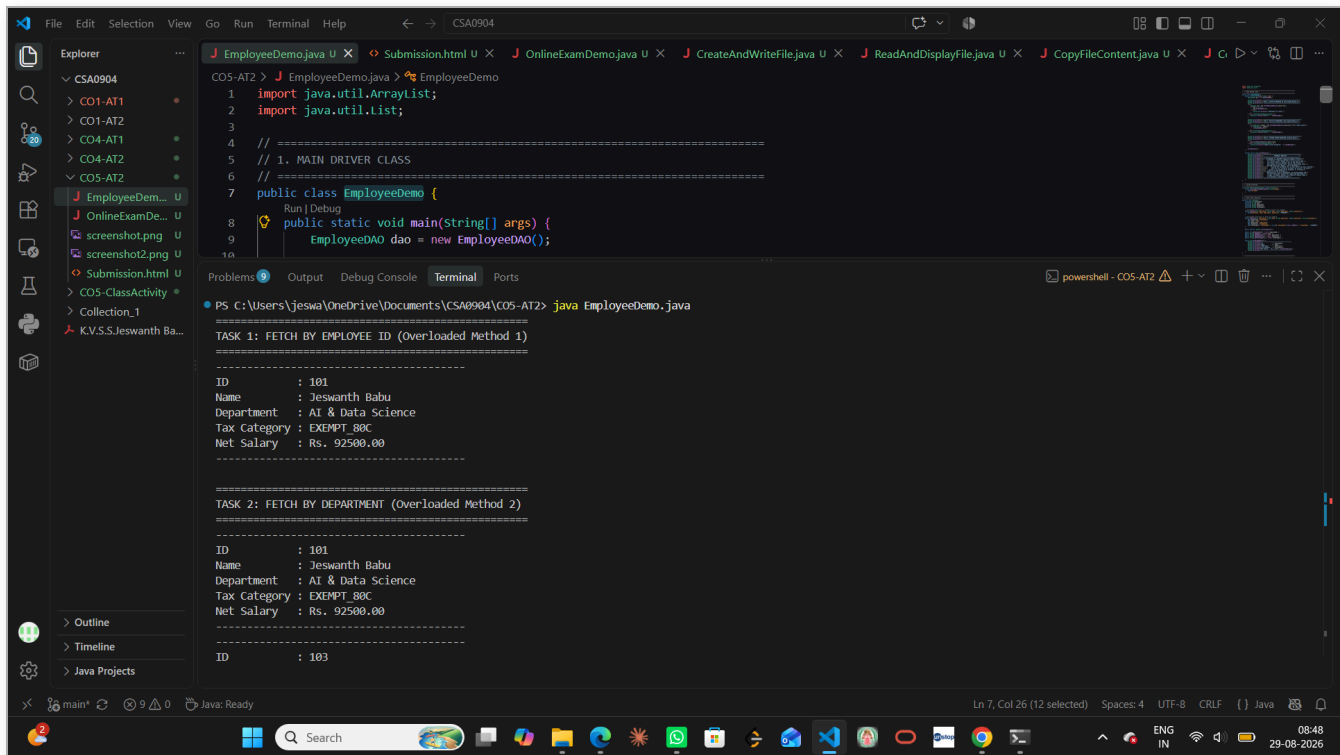
```

        if (rec.department.equalsIgnoreCase(department)) {
            employeeList.add(mapRecordToEmployee(rec));
        }
    }
    return employeeList;
}

private Employee mapRecordToEmployee(EmployeeRecord rec) throws InvalidSalaryDataException {
    if ("PERMANENT".equalsIgnoreCase(rec.empType)) {
        if (rec.baseSalary < 0) {
            throw new InvalidSalaryDataException("Negative base salary (" + rec.baseSalary + ") encountered for Employee ID: " + rec.empId);
        }
        return new PermanentEmployee(rec.empId, rec.name, rec.department, rec.baseSalary, rec.taxCategory, rec.bonus, rec.pfDeduction);
    } else if ("CONTRACT".equalsIgnoreCase(rec.empType)) {
        if (rec.hourlyRate <= 0 || rec.hoursWorked < 0) {
            throw new InvalidSalaryDataException("Invalid hourly rate (" + rec.hourlyRate + ") or hours (" + rec.hoursWorked + ") for Contract Employee ID: " + rec.empId);
        }
        return new ContractEmployee(rec.empId, rec.name, rec.department, rec.hourlyRate, rec.hoursWorked, rec.taxCategory);
    } else {
        throw new InvalidSalaryDataException("Unknown employee type '" + rec.empType + "' for Employee ID: " + rec.empId);
    }
}
}

```

3. OUTPUT EXECUTION SCREENSHOT



4. TERMINAL OUTPUT VERIFICATION

```
PS C:\Users\jeswa\OneDrive\Documents\CSA0904\CO5-AT2> java EmployeeDemo.java
```

```
=====
TASK 1: FETCH BY EMPLOYEE ID (Overloaded Method 1)
=====
```

```
-----
ID           : 101
Name          : Jeswanth Babu
Department    : AI & Data Science
Tax Category  : EXEMPT_80C
Net Salary    : Rs. 92500.00
-----
```

```
=====
TASK 2: FETCH BY DEPARTMENT (Overloaded Method 2)
=====
```

```
-----
ID           : 101
Name          : Jeswanth Babu
Department    : AI & Data Science
Tax Category  : EXEMPT_80C
Net Salary    : Rs. 92500.00
-----
```

```
-----
ID           : 103
Name          : Sai Charan
Department    : AI & Data Science
Tax Category  : STANDARD
Net Salary    : Rs. 102000.00
-----
```

```
=====
TASK 3: TESTING CUSTOM EXCEPTION (Invalid Data)
=====
```

Caught Custom Exception: Negative base salary (-5000.0) encountered for Employee ID: 105

=====

TECHNICAL ANALYSIS

=====

1. Incremental vs. Batched Processing (Midway Failure):

- Incremental Processing (Adding directly to list):
If a failure occurs midway (e.g., at row 250 of 500), the caller retains a partial list of 249 records. This exposes an inconsistent, partial application state.
- Batched Processing (Atomic / All-or-Nothing):
Records are validated into a local buffer and only returned once the full iteration succeeds. On failure, the local buffer is discarded and an exception is re-thrown.

2. Resource-Closing Strategy:

- Deterministic resource management (via AutoCloseable and try-with-resources) ensures all underlying descriptors, streams, and memory handles are closed in reverse order of allocation, preventing leaks.
- =====